Enterprise Employee RAG & Multi-Agent System

Complete solution understanding: how the system works, end to end

Part	What it answers
1. Executive overview	What the system is and what it can do
2. Technology stack	Which technology plays which role
3. Architecture	The four layers and how they connect
4. How a request is processed	The pipeline from a typed question to a streamed answer
5. Component reference	What each file and agent is responsible for
6. Intent routing	How the Supervisor decides which agent handles a query
7. Sequence diagram	Message-by-message interaction between components
8. Worked examples	Three real scenarios traced through the system
9. Policy ingestion	How policy documents become searchable
10. MCP server	Using the same capabilities from VS Code Copilot
11. Security and privacy	Guardrails and where data goes
12. Configuration and setup	LLM profiles and prerequisites
Appendix	How the system evolved across the two source documents

1. Executive overview

The **Enterprise Employee RAG & Multi-Agent System** lets people ask questions in plain English and get answers that draw on three different kinds of knowledge at once: **structured data** in Microsoft SQL Server (employees, departments, leave balances), **unstructured knowledge** in company policy documents (stored as vectors in ChromaDB), and **actions** such as drafting a ready-to-send email.

A team of specialised agents, coordinated by an orchestrator, decides which source or action each question needs. **LiteLLM** sits between the agents and the language model, so the chat model can be switched between a local **Ollama** model and the hosted **Google AI Studio Gemini** API by changing one setting. The whole system is exposed through a **FastAPI** backend and a light-theme web UI that streams answers token by token and shows a live trace of what each agent did.

What it can do

- **Answer policy questions** using retrieval-augmented generation (RAG) over uploaded policy documents.
- **Look up employee facts** such as profile, manager, and leave balances, and company-wide headcount.
- **Combine both**, e.g. "As per company leave policy, how many leaves can employee 2212 apply for?" needs the policy and that employee's balances.
- **Run ad-hoc analytics** by turning a question into a guarded, read-only T-SQL query.
- **Draft emails** with To, CC (manager), Subject and Body filled in from real employee data. Drafts are never sent automatically.
- **Remember the conversation**, so follow-ups like "email his manager" resolve correctly.

Capabilities delivered in four phases

Phase	Capability	Why it matters
1	Autonomous SQL tool calling	The LLM chooses among pre-approved, parameterized SQL functions instead of writing raw SQL for core lookups.
2	Conversational memory and pronoun resolution	Follow-up questions work naturally ("his manager", "that team").
3	Real-time SSE streaming	Answers appear progressively and the trace panel updates live.
4	Guarded text-to-SQL	Flexible analytics for questions the fixed tools do not cover, with strict safety checks.

2. Technology stack

Subsystem	Technology	Role in the solution
Backend framework	Python 3.14 + FastAPI	Asynchronous REST APIs, static UI hosting, Server-Sent Events (SSE) streaming.
Relational database	Microsoft SQL Server (MSSQL)	Primary enterprise store (EmployeeManagement DB): Employees, Departments, Designations, Leave Types and Balances, Chat Sessions and Messages.
Vector database	ChromaDB (persistent)	In-process vector store at ./vector_store holding policy chunk embeddings; cosine-similarity retrieval.
LLM gateway	LiteLLM (config.yaml)	Selectable *ollama* and *hosted* profiles, per-model routing, structured JSON output, streaming and tool calls.
Local AI services	Ollama	Local chat profile uses llama3.1 and qwen2.5:1.5b. Policy embeddings always use mxbai-embed-large locally, in either profile.
Hosted AI provider	Google AI Studio Gemini API	Hosted profile uses gemini-3.8-flash for default tasks and gemini-3.5-flash-lite for fast tasks. API key lives in the git-ignored config.local.yaml.
SQL safety	sqlglot	AST-level validation of generated T-SQL (SELECT only, row limit).
Frontend	HTML5 / CSS / vanilla JS	Light-theme SPA with live multi-agent trace viewer and progressive Markdown rendering.
Tool integration	MCP (stdio)	Local MCP server exposes employee, policy and email tools to VS Code Copilot Chat.

> **Key idea:** the database, the vector store and the policy embeddings always stay on your machine. Only the *chat model* is swappable between local and hosted.

3. Architecture

The system is organised into four layers. Requests travel down from the browser to the data layer and answers stream back up.

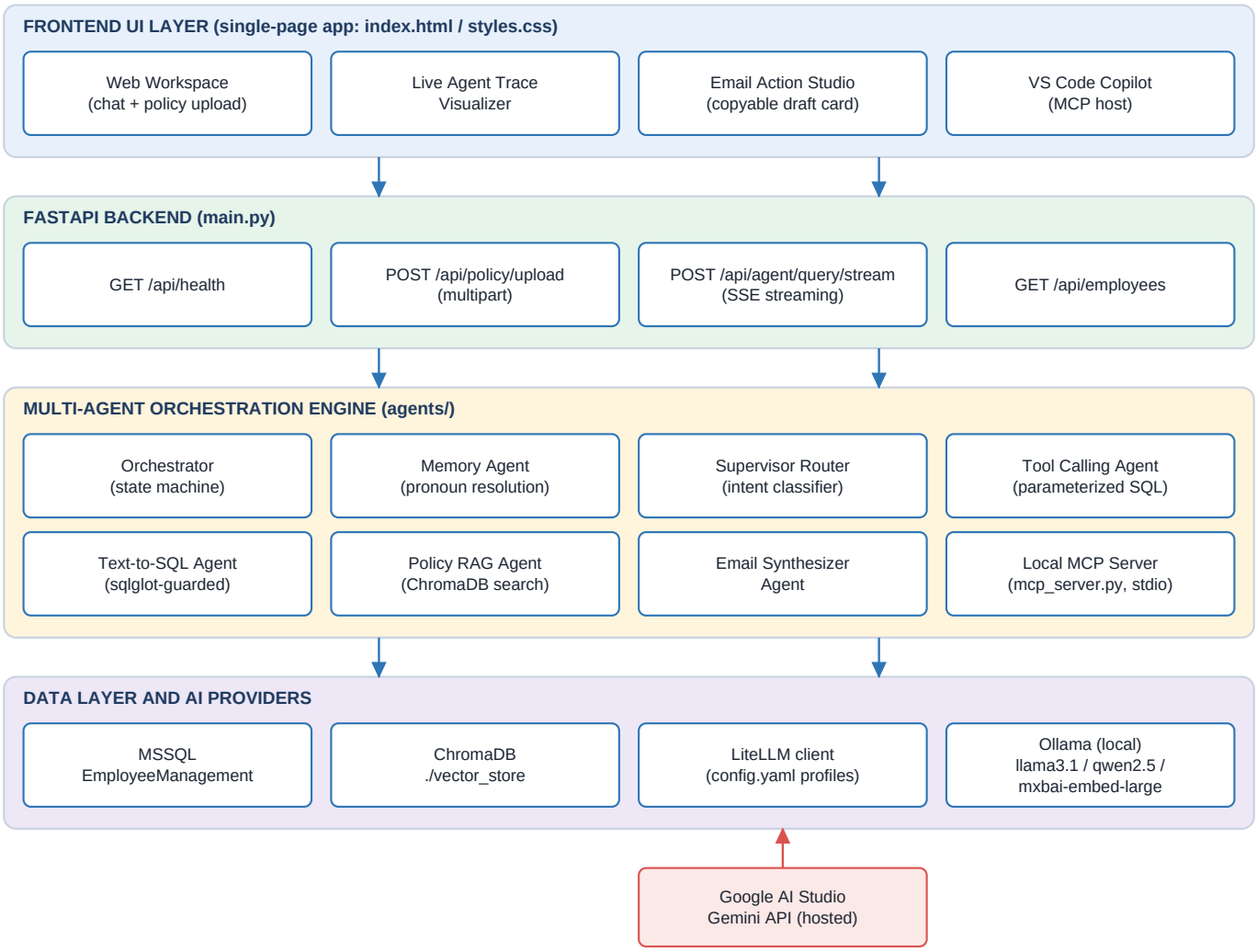

Figure 1. Layered architecture. Blue arrows show request flow; the red arrow shows the hosted Gemini option behind LiteLLM.

How the layers cooperate

- **Frontend** sends the query (or a policy upload) to the API and renders streamed tokens, the agent trace and any email draft.
- **FastAPI** validates requests and hands them to the orchestrator; it relays the orchestrator's generator output as SSE events.
- **Agent engine** is where decisions happen: rewrite the query, classify intent, run the right specialist, and synthesise the answer.
- **Data layer**: agents use pyodbc against MSSQL, cosine search against ChromaDB, and LiteLLM for every model call. LiteLLM forwards to Ollama or Gemini depending on `llm.active_profile`. Embeddings always go to Ollama.
- **MCP server** is a second entry point that reuses the same SQL tool executor, vector store and email agent for VS Code Copilot.

4. How a request is processed

Every question follows the same pipeline. Only the middle step (the execution track) changes depending on the intent.

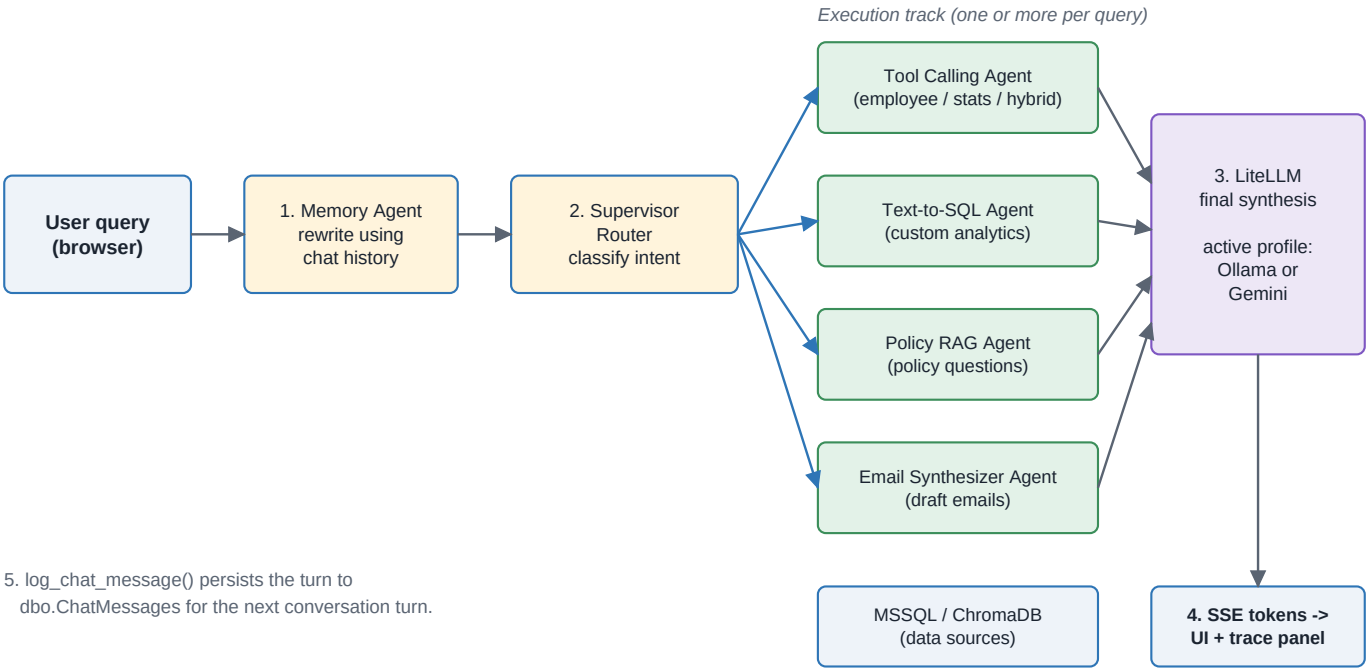

Figure 2. Request pipeline from user query to streamed answer.

Step by step

#	Stage	What happens
1	Query rewrite (Memory Agent)	Reads recent chat history from the database and rewrites the question into a standalone one, resolving pronouns such as "his manager". If the reference is ambiguous it asks the user to clarify instead of guessing.
2	Intent routing (Supervisor Router)	A fast LLM returns JSON containing the intent and any extracted entity such as an employee code (e.g. 2212).
3	Execution track	The orchestrator runs the matching specialist: Tool Calling Agent, Text-to-SQL Agent, Policy RAG Agent or Email Synthesizer Agent (details in section 6).
4	Synthesis and streaming	The LLM fuses retrieved data into one answer. The orchestrator yields `token` events that FastAPI streams to the browser as SSE; the UI renders Markdown progressively.
5	Persistence	`log_chat_message()` stores the turn in ChatMessages, giving the Memory Agent history for the next question.
6	Completion	A final `complete` event carries the result object and trace steps so the UI can finalise the trace panel and any email card.

> **Why an orchestrator as a generator?** Because it yields events as work progresses, the UI can show each agent step (rewrite, route, SQL, retrieval, synthesis) in real time instead of waiting for one final response.

5. Component reference

File	Component	Responsibility
main.py	FastAPI web server	REST endpoints (/api/agent/query, /api/agent/query/stream, /api/policy/upload, /api/health, /api/employees) and static SPA hosting.
database.py	MSSQL layer	pyodbc connectivity; queries Employees, Departments, Designations, LeaveTypes, EmployeeLeaveBalances, ChatSessions, ChatMessages; get_company_stats(); exposes a safe schema view (no Salary) for text-to-SQL.
vector_db.py	Vector store	ChromaDB persistence, recursive chunking (size 500, overlap 50), embeddings via Ollama mxbai-embed-large, cosine retrieval.
litellm_client.py config.yaml config.local.yaml	LLM client and profiles	Switchable Ollama / hosted profiles; per-model aliases (default and fast-llm); JSON output, streaming and tool calls. The local config holds the Gemini key and is git-ignored.
agents/memory_agent.py	Memory and Context Agent	Rewrites queries using chat history; resolves pronouns; requests clarification when needed.
agents/router_agent.py	Supervisor Router	Uses the fast-llm alias to classify intent and extract entities (e.g. employee code).
agents/tool_agent.py	Tool Calling Agent	Looping agent: the LLM requests pre-approved functions (e.g. get_employee_profile), the agent executes them via the parameterized SQL tool executor and feeds results back until the answer is ready.
agents/sql_agent.py	Employee SQL Agent	Retrieves employee profile, leave balances, manager email and company headcount; used by the tool layer.
agents/text_to_sql_agent.py	Text-to-SQL Agent	Generates T-SQL for ad-hoc analytics, validates it with sqlglot (SELECT only, TOP 50), executes read-only.
agents/policy_agent.py	Policy RAG Agent	Embeds the question, searches ChromaDB (top_k = 3) and returns policy context for synthesis.
agents/email_agent.py	Email Synthesizer Agent	Produces a structured email (To, CC manager, Subject, Body), normalising fields into a clean draft object.
agents/orchestrator.py	Multi-Agent Orchestrator	Coordinates the pipeline and yields trace and token events as a Python generator.
mcp_server.py	Local MCP server	Exposes the same capabilities as stdio MCP tools for VS Code Copilot.

6. Intent routing

The Supervisor Router classifies every query into one of six intents. Three of them share a single execution track, the Tool Calling Agent, which can call several data tools in a loop. That makes a hybrid question possible without a dedicated hybrid agent.

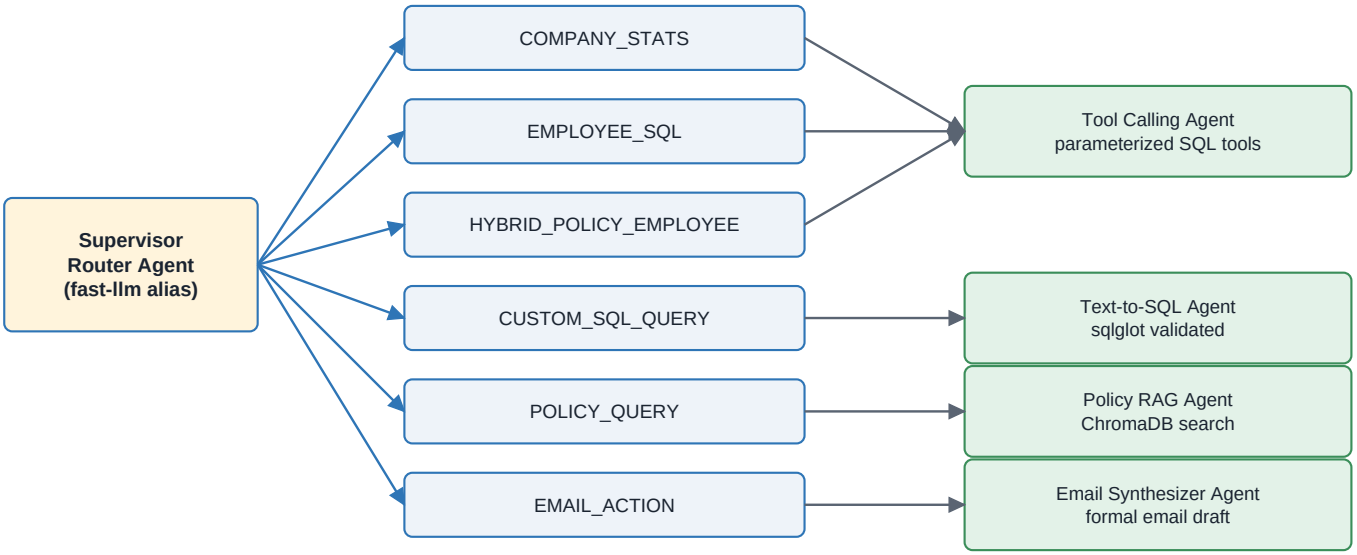

Figure 3. Intent to agent mapping.

Intent	Typical question	Handled by
COMPANY_STATS	How many employees do we have right now?	Tool Calling Agent (headcount / department stats tools)
EMPLOYEE_SQL	Show leave balance for employee 2212.	Tool Calling Agent (profile / leave tools)
HYBRID_POLICY_EMPLOYEE	As per policy, how many leaves can employee 2212 apply for?	Tool Calling Agent for employee data combined with policy context
CUSTOM_SQL_QUERY	Which department has the most people on leave? (not covered by a fixed tool)	Text-to-SQL Agent with sqlglot validation
POLICY_QUERY	What does our parental leave policy say?	Policy RAG Agent (ChromaDB search)
EMAIL_ACTION	Write an email to employee 2212 about training, CC his manager.	Email Synthesizer Agent, using employee and manager data

Tool calling vs text-to-SQL: core lookups use fixed, parameterized functions, which is safest and most predictable. Text-to-SQL is the fallback for open-ended analytics, and is wrapped in stricter validation.

7. End-to-end sequence diagram

The diagram condenses the full interaction into the order in which components talk to each other. Solid arrows are calls; dashed arrows are responses.

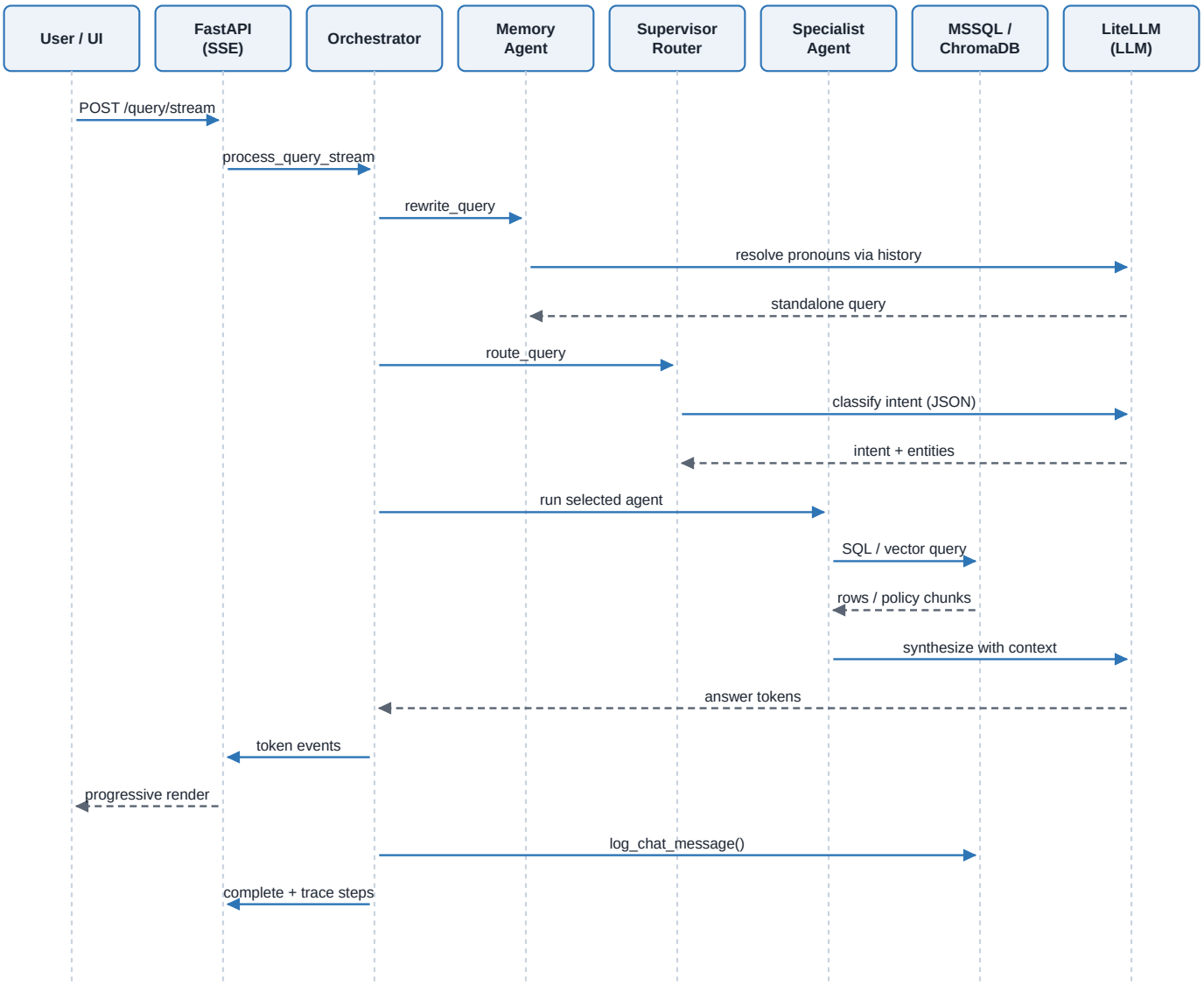

Figure 4. Message sequence for a typical streamed query. The specialist agent column stands for whichever track the router selected.

8. Worked examples

Workflow 1: policy plus employee balance (hybrid)

Prompt: *"As per company leave policy how many leaves employee 2212 can apply for?"*

- The router returns **HYBRID_POLICY_EMPLOYEE** with employee code 2212.
- The employee data is fetched from MSSQL: **CL = 8, EL = 18, SL = 10** days of balance.
- The policy agent embeds the question and retrieves the leave entitlement chunks from ChromaDB.
- LiteLLM fuses both contexts and calculates the total available leave: 8 + 18 + 10 = **36 days**, explaining it against the policy.

Workflow 2: automated email with manager in CC

Prompt: *"Write an email to employee 2212 to attend training keep his manager in cc."*

- The router returns **EMAIL_ACTION**.
- Employee data supplies the recipient (vikram.sharma2212@corpdemo.local) and manager (aarav.agarwal1@corpdemo.local).
- The Email Synthesizer Agent asks the LLM for structured JSON (subject, to, cc, body), then normalises the fields into a draft.
- The UI shows an interactive, copyable email card in the Email Action Studio. Nothing is sent.

Workflow 3: company headcount

Prompt: *"How many employees do we have in our company right now?"*

- The router returns **COMPANY_STATS**.
- The SQL tooling queries dbo.Employees and dbo.Departments.
- The answer reports **201 active employees across 10 departments**.

Workflow 4: follow-up with memory

Prompts: *"Show employee 2212's profile."* then *"Email his manager about the deadline."*

- For the second prompt the Memory Agent reads history and rewrites it to a standalone query that names employee 2212's manager.
- Routing and drafting then proceed as in Workflow 2. If the pronoun could not be resolved, the agent asks which employee is meant.

9. Policy ingestion

Policy documents (PDF, Markdown or TXT) become searchable through the upload endpoint and the vector store.

Step	Action
1	The user uploads a document through the UI; it is sent as multipart data to `POST /api/policy/upload`.
2	The text is extracted and split with recursive character splitting: chunk size 500, overlap 50, so context is not cut mid-idea.
3	Each chunk is embedded locally with Ollama **mxbai-embed-large**.
4	Vectors and chunk text are stored in ChromaDB at `./vector_store` (persistent).
5	At question time the query is embedded the same way, and the top 3 chunks by cosine similarity are returned as policy context.

> Because embeddings always run on local Ollama, policy text is never sent to a hosted provider for indexing, even when the hosted chat profile is active. Retrieved chunks are, however, included in the prompt sent to the chat model (see section 11).

10. MCP server and VS Code usage

`mcp_server.py` exposes the existing capabilities as local MCP tools over stdio. VS Code discovers it through `.vscode/mcp.json`, which launches the server with the workspace `.venv` interpreter. It reuses the parameterized SQL tool executor, VectorDBManager and the email agent.

Tool	Purpose
get_employee_profile	Employee profile details
get_leave_balance	Leave balances for an employee
get_team_members	Team for a given manager
get_department_stats	Department and headcount statistics
search_policy	Semantic search over policy documents
draft_employee_email	Returns a draft email; does not send it

To use it

- Install dependencies: `pip install -r requirements.txt`.
- In VS Code open the Command Palette, run **MCP: List Servers**, and start **employee-management-rag**.
- In Copilot Chat, enable the server's tools in the tools picker.
- Try: "Get employee 2212's leave balance", "Show the team for manager 1001", "Search company policy for parental leave", "Draft an email to employee 2212 about training".

> **Prerequisites:** SQL tools need SQL Server running; policy search needs Ollama with mxbai-embed-large; email drafting uses the active LLM profile.

11. Security and privacy

Control	How it works
Configurable data locality	With *active_profile: ollama*, chat inference and embeddings run entirely locally. With *active_profile: hosted*, chat prompts plus the employee and policy context supplied to the model are sent to Google AI Studio. Policy embeddings and ChromaDB stay local either way.
Secret handling	The hosted API key is stored in git-ignored config.local.yaml, not in the committed config.yaml.
Read-only SQL	Generated T-SQL is parsed into an AST with sqlglot; destructive statements (DROP, UPDATE, INSERT and similar) are rejected.
Exfiltration limits	Text-to-SQL forces a TOP 50 limit on arbitrary queries, and Salary is excluded from the schema shown to the LLM.
Parameterized tools	The Tool Calling Agent only executes pre-approved Python functions with strict Pydantic definitions, removing classic SQL injection paths from core functionality.
Drafts only	Email generation (UI and MCP) creates a draft for a human to review; nothing is sent automatically.

Practical takeaway: pick the *ollama* profile when employee data must not leave the machine, and the *hosted* profile when answer quality or speed matters more. Switching is a one-line change.

12. Configuration and setup

LLM profiles

Setting	Value	Effect
llm.active_profile	hosted or ollama	Selects which provider LiteLLM routes to.
Hosted: default model	gemini-3.8-flash	Main reasoning, synthesis and tool-calling tasks.
Hosted: fast model	gemini-3.5-flash-lite	Quick tasks such as intent routing.
Ollama: models	llama3.1, qwen2.5:1.5b	Local default and fast models.
Embeddings	mxbai-embed-large (Ollama)	Always local, independent of the chat profile.
Gemini API key	config.local.yaml	Git-ignored; required only for the hosted profile.

Prerequisites

- Python environment with `pip install -r requirements.txt`.
- Microsoft SQL Server on localhost with the EmployeeManagement database and an ODBC driver for pyodbc.
- Ollama running with mxbai-embed-large pulled (plus llama3.1 and qwen2.5:1.5b for the local chat profile).
- For the hosted profile: a Google AI Studio API key in config.local.yaml.

Appendix: how the design evolved across the two source documents

The two uploaded files describe the same system at two points in its development. This PDF follows the newer document (SOLUTION_ARCHITECTURE.md) as the primary reference and uses the older one (architecture_report.md) for the concrete workflow examples.

Aspect	architecture_report.md (earlier)	SOLUTION_ARCHITECTURE.md (current)
Query preparation	Query goes straight to the router.	Memory Agent rewrites the query first (pronoun resolution, clarification).
SQL access	Orchestrator calls the Employee SQL Agent directly.	Tool Calling Agent lets the LLM invoke parameterized SQL tools in a loop.
Ad-hoc analytics	Not present.	Text-to-SQL Agent with sqlglot validation.
Intents	COMPANY_STATS_QUERY, POLICY_QUERY, EMPLOYEE_SQL_QUERY, HYBRID_POLICY_EMPLOYEE, EMAIL_ACTION.	COMPANY_STATS, EMPLOYEE_SQL, HYBRID_POLICY_EMPLOYEE, CUSTOM_SQL_QUERY, POLICY_QUERY, EMAIL_ACTION.
Response delivery	Single response JSON with answer, email draft and trace steps.	SSE stream of token events followed by a complete event.
Documentation focus	Component table, sequence flow and three workflows.	Stack, security constraints, routing logic, MCP tool list, delivered phases.

> The earlier document's three workflows remain valid under the current design. The only difference is the internal path: employee lookups now go through the Tool Calling Agent rather than a direct SQL Agent call.